

Problema de árbol generador mínimo

Programación III - UNGS

Árboles

- Un **árbol** es un grafo conexo sin circuitos.

Árboles

- Un **árbol** es un grafo conexo sin circuitos.
- **Teorema.** Dado un grafo $G = (V, E)$, son equivalentes:
 1. G es un árbol.
 2. G es un grafo sin circuitos simples, pero si se agrega cualquier arista e a G resulta un grafo con exactamente un circuito simple, y ese circuito contiene a e .

Árboles

- Un **árbol** es un grafo conexo sin circuitos.
- **Teorema.** Dado un grafo $G = (V, E)$, son equivalentes:
 1. G es un árbol.
 2. G es un grafo sin circuitos simples, pero si se agrega cualquier arista e a G resulta un grafo con exactamente un circuito simple, y ese circuito contiene a e .
 3. Existe exactamente un camino simple entre todo par de vértices.

Árboles

- Un **árbol** es un grafo conexo sin circuitos.
- **Teorema.** Dado un grafo $G = (V, E)$, son equivalentes:
 1. G es un árbol.
 2. G es un grafo sin circuitos simples, pero si se agrega cualquier arista e a G resulta un grafo con exactamente un circuito simple, y ese circuito contiene a e .
 3. Existe exactamente un camino simple entre todo par de vértices.
 4. G es conexo, pero si se quita cualquier arista a G queda un grafo no conexo.

Árboles

- Una **hoja** de un árbol es un vértice de grado 1.

Árboles

- Una **hoja** de un árbol es un vértice de grado 1.
- **Teorema.** Todo árbol no trivial (es decir, con al menos dos vértices) tiene al menos dos hojas.

Árboles

- Una **hoja** de un árbol es un vértice de grado 1.
- **Teorema.** Todo árbol no trivial (es decir, con al menos dos vértices) tiene al menos dos hojas.
- **Teorema.** Si $G = (V, E)$ es un árbol, entonces $|E| = |V| - 1$.

Árbol generador mínimo

- Dado un grafo G , un **árbol generador** de G es un subgrafo de G que es un árbol y tiene el mismo conjunto de vértices que G .

Árbol generador mínimo

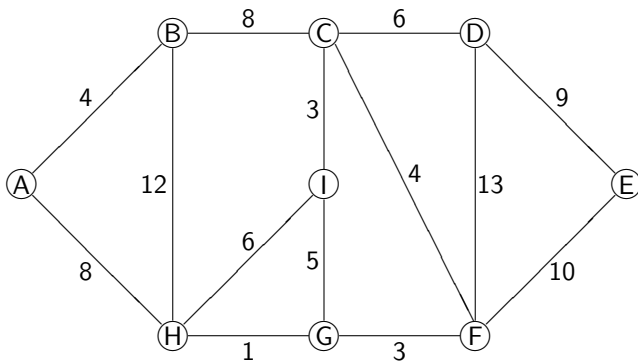
- Dado un grafo G , un **árbol generador** de G es un subgrafo de G que es un árbol y tiene el mismo conjunto de vértices que G .
- Sea $T = (V, E)$ un árbol y $l : E \rightarrow R$ una función que asigna longitudes (o pesos) a las aristas de T . Se define la **longitud** de T como $l(T) = \sum_{e \in E} l(e)$.

Árbol generador mínimo

- Dado un grafo G , un **árbol generador** de G es un subgrafo de G que es un árbol y tiene el mismo conjunto de vértices que G .
- Sea $T = (V, E)$ un árbol y $l : E \rightarrow R$ una función que asigna longitudes (o pesos) a las aristas de T . Se define la **longitud** de T como $l(T) = \sum_{e \in E} l(e)$.
- Dado un grafo $G = (V, E)$ un **árbol generador mínimo** T de G es un árbol generador de G de mínima longitud, es decir

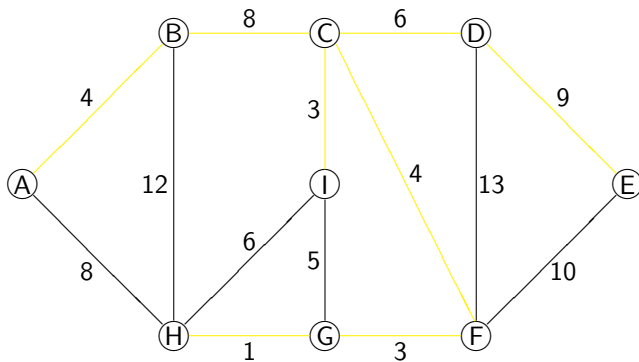
$$l(T) \leq l(T') \quad \forall T' \text{ árbol generador de } G.$$

Ejemplo



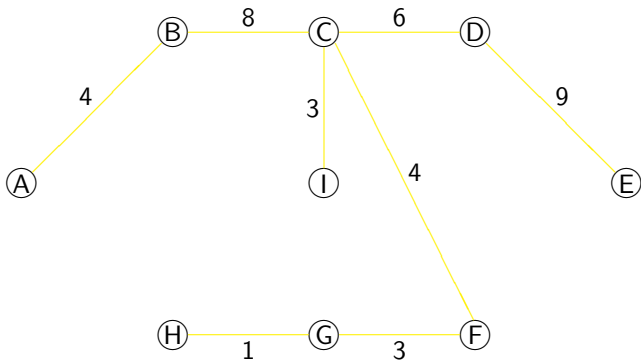


Ejemplo

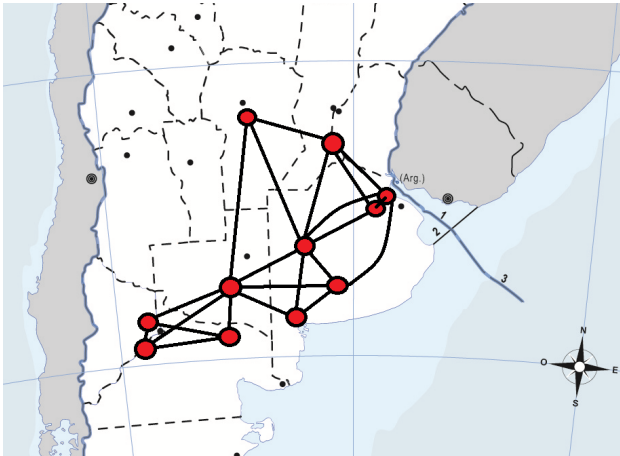




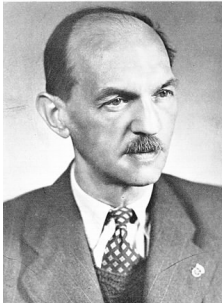
Ejemplo



Aplicación: Diseño de redes eléctricas



Algoritmo de Prim



Vojtech Jarník
(1897–1970)



Robert Prim
(1921)



Edsger Dijkstra
(1930–2002)

Algoritmo de Prim

$V_T := \{u\}$ (u cualquier vértice de G)

$E_T := \emptyset$

$i := 1$

mientras $i \leq n - 1$ **hacer**

elegir $e = (u, v) \in E$ tal que $l(e)$ sea mínima
entre las aristas que tienen un extremo

$u \in V_T$ y el otro $v \in V \setminus V_T$

$E_T := E_T \cup \{e\}$

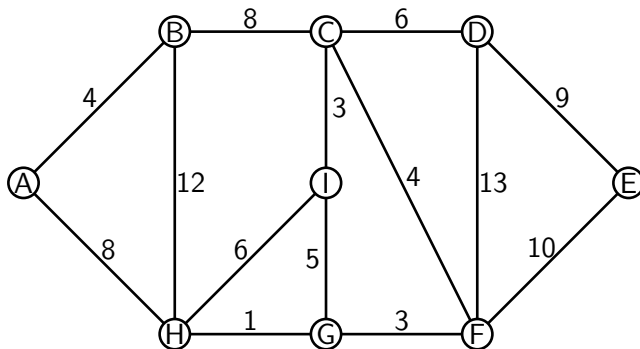
$V_T := V_T \cup \{v\}$

$i := i + 1$

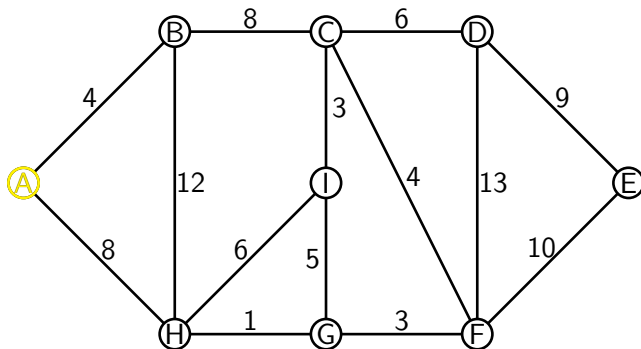
retornar $T = (V_T, E_T)$



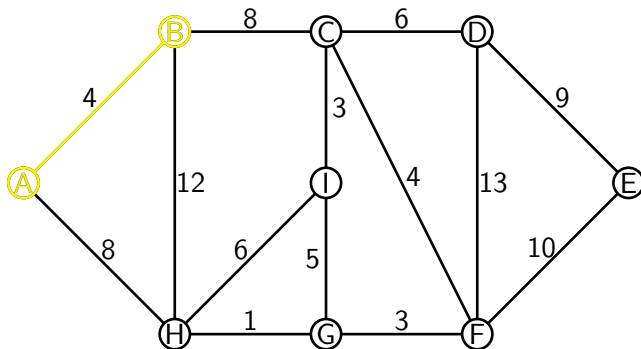
Ejemplo



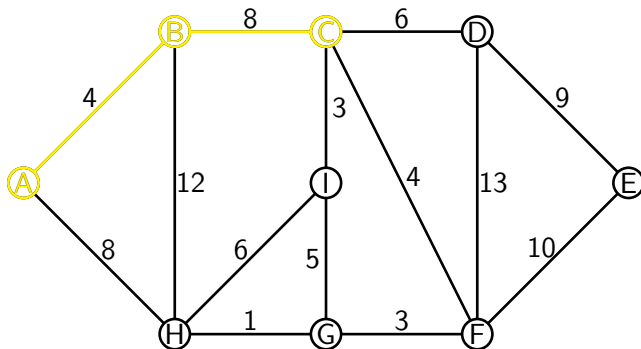
Ejemplo



Ejemplo

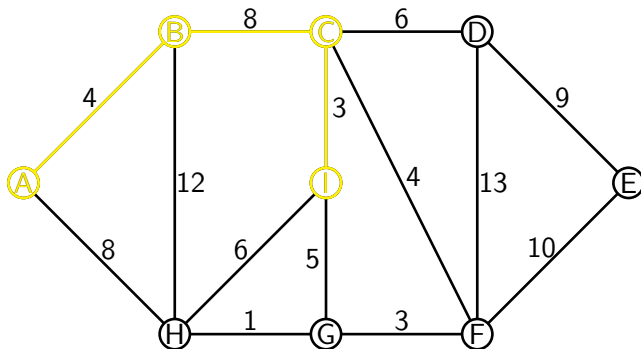


Ejemplo

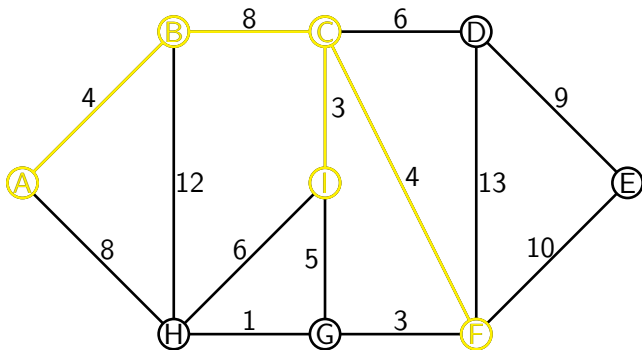




Ejemplo

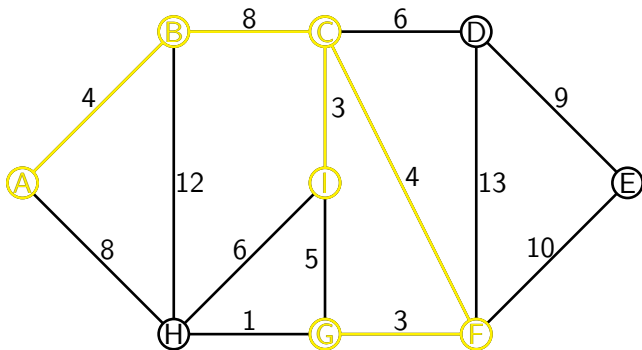


Ejemplo



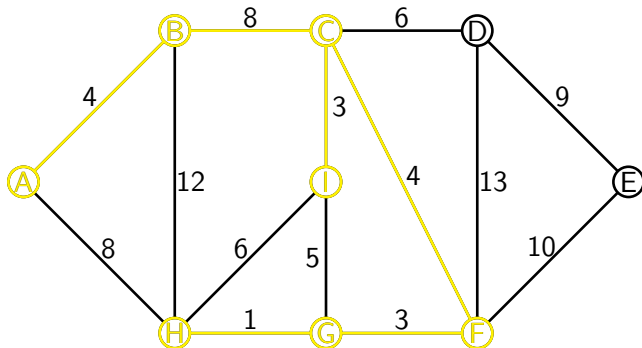


Ejemplo



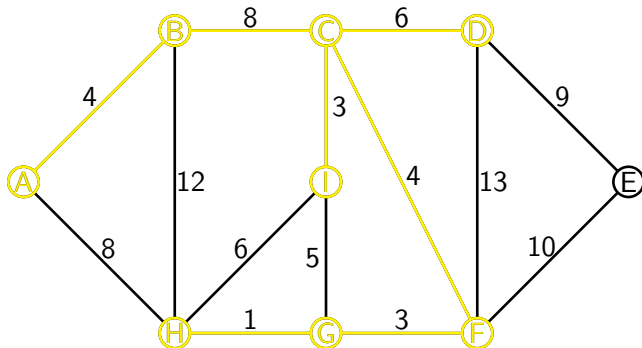


Ejemplo



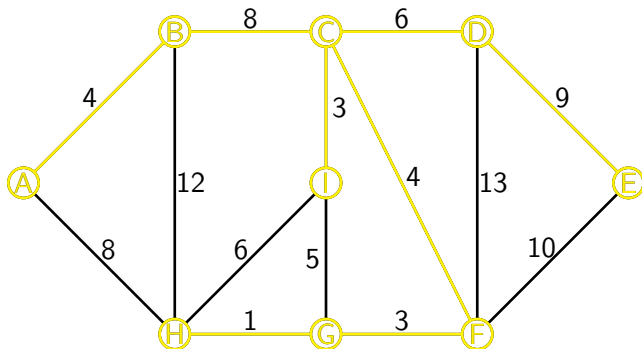


Ejemplo





Ejemplo



Algoritmo de Prim

- **Teorema.** Sea F un subconjunto de aristas en un AGM de G , y sea S un conjunto de vértices de una componente conexa de F . Sea ij una arista de peso mínimo entre S y $\bar{S}$. Entonces, existe un AGM de G que incluye a todas las aristas de $F \cup \{ij\}$.

Algoritmo de Prim

- **Teorema.** Sea F un subconjunto de aristas en un AGM de G , y sea S un conjunto de vértices de una componente conexa de F . Sea ij una arista de peso mínimo entre S y $\bar{S}$. Entonces, existe un AGM de G que incluye a todas las aristas de $F \cup \{ij\}$.
- **Teorema.** El algoritmo de Prim es **correcto**. Es decir, dado un grafo G conexo, determina un árbol generador mínimo de G .

Algoritmo de Prim

- **Teorema.** Sea F un subconjunto de aristas en un AGM de G , y sea S un conjunto de vértices de una componente conexa de F . Sea ij una arista de peso mínimo entre S y $\bar{S}$. Entonces, existe un AGM de G que incluye a todas las aristas de $F \cup \{ij\}$.
- **Teorema.** El algoritmo de Prim es **correcto**. Es decir, dado un grafo G conexo, determina un árbol generador mínimo de G .
- ¿Cuál es la complejidad computacional de este algoritmo?

Algoritmo de Kruskal



Joseph Kruskal (1928–2010)

Algoritmo de Kruskal

$E_T := \emptyset$

$i := 1$

mientras $i \leq n - 1$ **hacer**

elegir $e \in E$ tal que $l(e)$ sea mínima entre las
aristas que no forman circuito con las
aristas que ya están en E_T

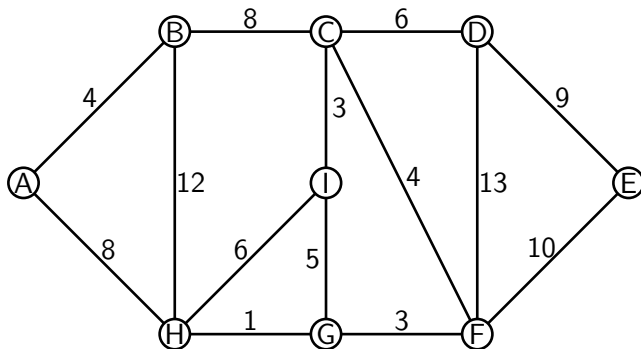
$E_T := E_T \cup \{e\}$

$i := i + 1$

retornar $T = (V, E_T)$

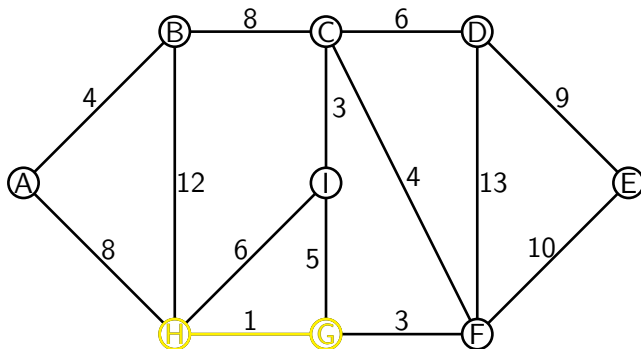


Ejemplo



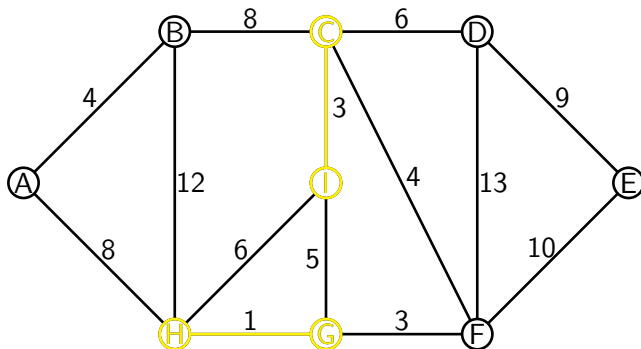


Ejemplo



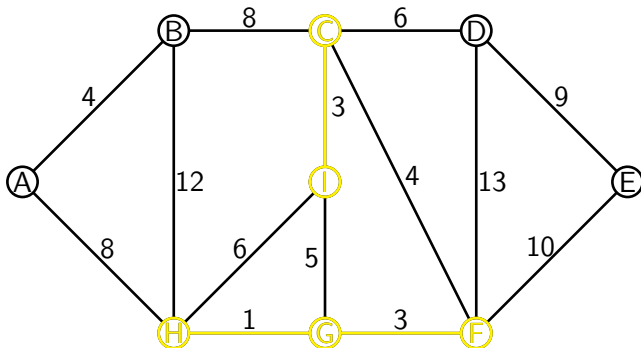


Ejemplo



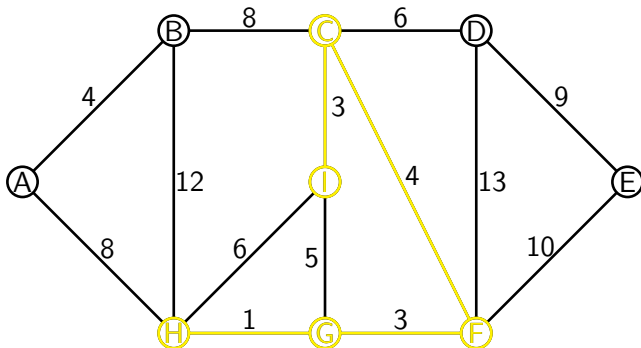


Ejemplo



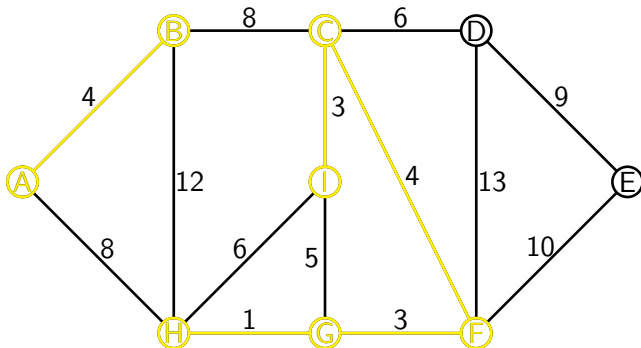


Ejemplo



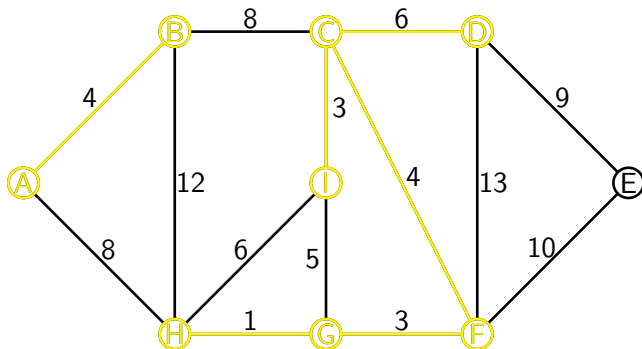


Ejemplo



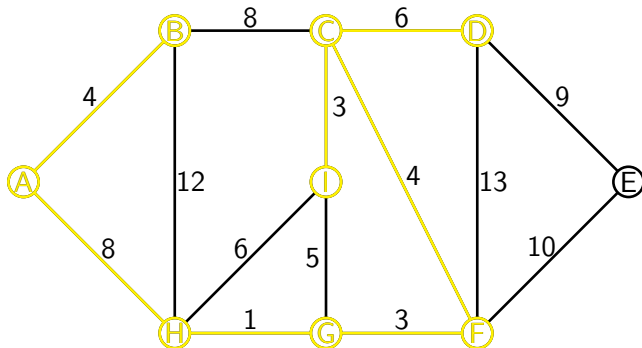


Ejemplo



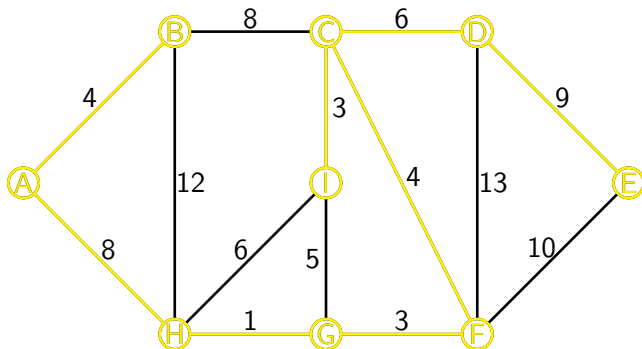


Ejemplo





Ejemplo



Union-Find



Bernard Galler (1928–2006)



Michael Fischer (1942)

- Podemos mejorar la complejidad computacional del Algoritmo de Kruskal por medio de la estructura de datos **union-find** (Galler y Fischer, 1964).

Union-Find

- Mantenemos un arreglo especificando en qué componente conexa está cada vértice.

Union-Find

- Mantenemos un arreglo especificando en qué componente conexa está cada vértice.
 1. El arreglo contiene el **padre** de cada vértice. Cada vértice está en la misma componente conexa que su padre.

Union-Find

- Mantenemos un arreglo especificando en qué componente conexa está cada vértice.
 1. El arreglo contiene el **padre** de cada vértice. Cada vértice está en la misma componente conexa que su padre.
 2. Se forma una estructura de **árbol**, cuya raíz es el **representante** de la componente conexa.

Union-Find

- Mantenemos un arreglo especificando en qué componente conexa está cada vértice.
 1. El arreglo contiene el **padre** de cada vértice. Cada vértice está en la misma componente conexa que su padre.
 2. Se forma una estructura de **árbol**, cuya raíz es el **representante** de la componente conexa.
- Inicialmente, cada vértice está en su propia componente conexa:

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

Union-Find

- Después de algunas iteraciones, tendremos varias componentes conexas:

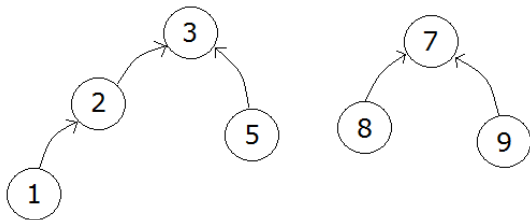
0	2	3	3	4	3	6	7	7	7
---	---	---	---	---	---	---	---	---	---

Union-Find

- Después de algunas iteraciones, tendremos varias componentes conexas:

0	2	3	3	4	3	6	7	7	7
---	---	---	---	---	---	---	---	---	---

- Representamos esta situación con un árbol por cada componente conexas:



Union-Find

- Dado un vértice, es fácil encontrar su raíz:

```

1  int root(int i)
2  {
3      while(A[i] != i)
4          i = A[i];
5
6      return i;
7  }
```

Union-Find

- Dado un vértice, es fácil encontrar su raíz:

```

1  int root(int i)
2  {
3      while(A[i] != i)
4          i = A[i];
5
6      return i;
7  }
```

- Para determinar si dos vértices están en la misma componente conexa:

```

1  boolean find(int i, int j)
2  {
3      return raiz(i) == raiz(j);
4  }
```

Union-Find

- La única operación que modifica la estructura es la **unión** de dos componentes conexas.

Union-Find

- La única operación que modifica la estructura es la **unión** de dos componentes conexas.
- Hacemos que la raíz de una de ellas apunte a la raíz de la otra:

```

1 void union(int i, int j)
2 {
3     int ri = root(i);
4     int rj = root(j);
5
6     A[ri] = rj;
7 }
```

Union-Find

- La única operación que modifica la estructura es la **unión** de dos componentes conexas.
- Hacemos que la raíz de una de ellas apunte a la raíz de la otra:

```

1 void union(int i, int j)
2 {
3     int ri = root(i);
4     int rj = root(j);
5
6     A[ri] = rj;
7 }
```

- ¿Qué complejidad computacional tienen estas operaciones?

Union-Find

- La única operación que modifica la estructura es la **unión** de dos componentes conexas.
- Hacemos que la raíz de una de ellas apunte a la raíz de la otra:

```

1 void union(int i, int j)
2 {
3     int ri = root(i);
4     int rj = root(j);
5
6     A[ri] = rj;
7 }
```

- ¿Qué complejidad computacional tienen estas operaciones?
1. Union:

Union-Find

- La única operación que modifica la estructura es la **unión** de dos componentes conexas.
- Hacemos que la raíz de una de ellas apunte a la raíz de la otra:

```

1 void union(int i, int j)
2 {
3     int ri = root(i);
4     int rj = root(j);
5
6     A[ri] = rj;
7 }
```

- ¿Qué complejidad computacional tienen estas operaciones?
1. Union: $O(n)$

Union-Find

- La única operación que modifica la estructura es la **unión** de dos componentes conexas.
- Hacemos que la raíz de una de ellas apunte a la raíz de la otra:

```

1 void union(int i, int j)
2 {
3     int ri = root(i);
4     int rj = root(j);
5
6     A[ri] = rj;
7 }
```

- ¿Qué complejidad computacional tienen estas operaciones?
1. Union: $O(n)$
 2. Find:

Union-Find

- La única operación que modifica la estructura es la **unión** de dos componentes conexas.
- Hacemos que la raíz de una de ellas apunte a la raíz de la otra:

```

1 void union(int i, int j)
2 {
3     int ri = root(i);
4     int rj = root(j);
5
6     A[ri] = rj;
7 }
```

- ¿Qué complejidad computacional tienen estas operaciones?
- Union: $O(n)$
 - Find: $O(n)$

Union-Find

- Si llevamos la cuenta de la cantidad de elementos en cada componente conexas, podemos hacer que la raíz del menor árbol apunte a la raíz del mayor.

Union-Find

- Si llevamos la cuenta de la cantidad de elementos en cada componente conexas, podemos hacer que la raíz del menor árbol apunte a la raíz del mayor. Con esto, logramos:

1. Union:

Union-Find

- Si llevamos la cuenta de la cantidad de elementos en cada componente conexas, podemos hacer que la raíz del menor árbol apunte a la raíz del mayor. Con esto, logramos:
 1. Union: $O(\log n)$
 2. Find:

Union-Find

- Si llevamos la cuenta de la cantidad de elementos en cada componente conexas, podemos hacer que la raíz del menor árbol apunte a la raíz del mayor. Con esto, logramos:
 1. Union: $O(\log n)$
 2. Find: $O(\log n)$

Union-Find

- Si llevamos la cuenta de la cantidad de elementos en cada componente conexa, podemos hacer que la raíz del menor árbol apunte a la raíz del mayor. Con esto, logramos:
 1. Union: $O(\log n)$
 2. Find: $O(\log n)$
- Podemos **compactar** los árboles cuando se hace `find()`, con las siguientes operaciones:

Union-Find

- Si llevamos la cuenta de la cantidad de elementos en cada componente conexa, podemos hacer que la raíz del menor árbol apunte a la raíz del mayor. Con esto, logramos:
 1. Union: $O(\log n)$
 2. Find: $O(\log n)$
- Podemos **compactar** los árboles cuando se hace `find()`, con las siguientes operaciones:
 1. **Path compression:** Hacer que todos los nodos apunten a la raíz.

Union-Find

- Si llevamos la cuenta de la cantidad de elementos en cada componente conexa, podemos hacer que la raíz del menor árbol apunte a la raíz del mayor. Con esto, logramos:
 1. Union: $O(\log n)$
 2. Find: $O(\log n)$
- Podemos **compactar** los árboles cuando se hace `find()`, con las siguientes operaciones:
 1. **Path compression:** Hacer que todos los nodos apunten a la raíz.
 2. **Path splitting:** Hacer que cada nodo apunte a su abuelo.

Union-Find

- Si llevamos la cuenta de la cantidad de elementos en cada componente conexa, podemos hacer que la raíz del menor árbol apunte a la raíz del mayor. Con esto, logramos:
 1. Union: $O(\log n)$
 2. Find: $O(\log n)$
- Podemos **compactar** los árboles cuando se hace `find()`, con las siguientes operaciones:
 1. **Path compression:** Hacer que todos los nodos apunten a la raíz.
 2. **Path splitting:** Hacer que cada nodo apunte a su abuelo.
 3. **Path halving:** Aplicar *path splitting* nodo de por medio.

Union-Find

- **Definición.** Función de Ackermann:

$$A(m, n) = \begin{cases} n + 1 & \text{si } m = 0 \\ A(m - 1, 1) & \text{si } m > 0 \text{ y } n = 0 \\ A(m - 1, A(m, n - 1)) & \text{si } m > 0 \text{ y } n > 0 \end{cases}$$

Union-Find

- **Definición.** Función de Ackermann:

$$A(m, n) = \begin{cases} n + 1 & \text{si } m = 0 \\ A(m - 1, 1) & \text{si } m > 0 \text{ y } n = 0 \\ A(m - 1, A(m, n - 1)) & \text{si } m > 0 \text{ y } n > 0 \end{cases}$$

- La función de Ackermann crece muy rápidamente! Por ejemplo, $A(4, 2)$ tiene 19.729 dígitos decimales.

Union-Find

- **Definición.** Función de Ackermann:

$$A(m, n) = \begin{cases} n + 1 & \text{si } m = 0 \\ A(m - 1, 1) & \text{si } m > 0 \text{ y } n = 0 \\ A(m - 1, A(m, n - 1)) & \text{si } m > 0 \text{ y } n > 0 \end{cases}$$

- La función de Ackermann crece muy rápidamente! Por ejemplo, $A(4, 2)$ tiene 19.729 dígitos decimales.
- Si se usa alguna de las técnicas de compactación y la unión por tamaño (o por altura), entonces el **tiempo amortizado** de cada operación es $O(\alpha(n))$, donde $\alpha(n)$ es la inversa de $A(n, n)$.

Union-Find

- **Definición.** Función de Ackermann:

$$A(m, n) = \begin{cases} n + 1 & \text{si } m = 0 \\ A(m - 1, 1) & \text{si } m > 0 \text{ y } n = 0 \\ A(m - 1, A(m, n - 1)) & \text{si } m > 0 \text{ y } n > 0 \end{cases}$$

- La función de Ackermann crece muy rápidamente! Por ejemplo, $A(4, 2)$ tiene 19.729 dígitos decimales.
- Si se usa alguna de las técnicas de compactación y la unión por tamaño (o por altura), entonces el **tiempo amortizado** de cada operación es $O(\alpha(n))$, donde $\alpha(n)$ es la inversa de $A(n, n)$.
- La función $\alpha(n)$ crece muy lentamente! A fines prácticos, $\alpha(n) \leq 5$.